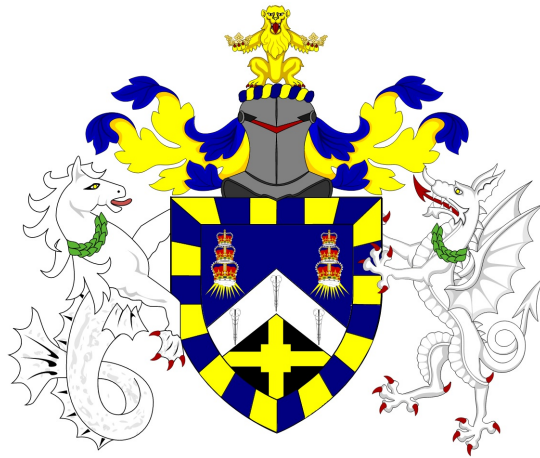


Data Analytics MSc Dissertation MTHM038, 2024/25

Comparative Analysis of ARIMA, LSTM, Transformer, and Prophet Models for Stock Price Forecasting

Ji Yoon Moon, ID 240145156

Supervisor: Prof. Navid Nabijou



A thesis presented for the degree of
Master of Science in *Data Analytics*

School of Mathematical Sciences
Queen Mary University of London

Declaration of original work

Student's Declaration: I Jiyeon Moon by declare that the work in this thesis is my original work. I have not copied from any other students' work, work of mine submitted elsewhere, or from any other sources except where due reference or acknowledgement is made explicitly in the text. Furthermore, no part of this dissertation has been written for me by another person, by generative artificial intelligence (AI), or by AI-assisted technologies.

Referenced text has been flagged by:

1. Using italic fonts, **and**
2. using quotation marks "...", **and**
3. explicitly mentioning the source in the text.

Abstract

Accurately predicting stock prices is essential for financial decision-making, but it remains a challenging task due to market fluctuations, nonlinearity, and external factors. This paper compares ARIMA (Autoregressive Integrated Moving Average), LSTM (Long Short-Term Memory), Transformer, and Prophet models for predicting daily closing prices of Amazon (AMZN), Microsoft (MSFT), and ExxonMobil (XOM) stocks. These models represent diverse industries, including e-commerce, technology, and energy. Using Yahoo Finance data from January 1, 2022, to January 1, 2025, the paper evaluates the performance of 1-day and 3-day forward predictions using a working forward validation method with a rolling window. The data preprocessing process included outlier handling using winsorization, normalization, and exploratory analysis to identify trends and volatility. The model was implemented using optimized hyperparameters. ARIMA employed AIC-based order selection, LSTM and Transformer utilized sequence-based architectures, and Prophet accounted for seasonality and holiday factors. To evaluate performance, the models were tested on unseen data using MAE, MAPE, and RMSE metrics. According to the results, ARIMA and Prophet models generally outperformed LSTM and Transformer, with ARIMA recording the lowest error across all stocks, especially in stable series such as XOM. The Prophet model excelled at capturing seasonal patterns, while LSTM and Transformer exhibited higher errors, particularly in volatile stocks like AMZN. However, when the forecast horizon was extended from one day to three days, LSTM

and Transformer exhibited smaller increases in error compared to ARIMA and Prophet, which showed significantly larger error growth. In conclusion, statistical models including ARIMA and Prophet, are more effective for short-term and stable financial forecasting. In contrast, neural network models like LSTM and Transformer showed lower error increase rates as the prediction period extended, despite their higher baseline errors. Additionally, this study found that industry-specific volatility and prediction period influence prediction performance. Future research could improve model performance by incorporating external variables or hybrid models.

Contents

1	Introduction	6
1.1	Problem Statement and Motivation	6
1.2	Research Objectives and Questions	6
2	Literature Review	8
2.1	Related Work	8
2.2	Summary	11
3	Methodology	12
3.1	Data Preprocessing	12
3.2	Model Selection	13
3.2.1	ARIMA	13
3.2.2	LSTM	16
3.2.3	Transformer	18
3.2.4	Prophet	21
3.3	Model Evaluation Metrics	24
4	Results and Discussion	27
4.1	Dataset	27
4.2	Data Preprocessing	27
4.3	Exploratory Data Analysis	29
4.4	ARIMA Model Results	30
4.4.1	Model Fitting Procedure	30

4.4.2	1-Day Ahead Forecasting	30
4.4.3	3-Day Ahead Forecasting	34
4.5	LSTM Model Results	37
4.5.1	Model Fitting Procedure	37
4.5.2	1-Day Ahead Forecasting	38
4.5.3	3-Day Ahead Forecasting	40
4.5.4	Discussion	43
4.6	Transformer Model Results	44
4.6.1	Model Fitting Procedure	44
4.6.2	1-Day Ahead Forecasting	45
4.6.3	3-Day Ahead Forecasting	47
4.6.4	Discussion	49
4.7	Prophet Model Results	50
4.7.1	Model Fitting Procedure	50
4.7.2	1-Day Ahead Forecasting	50
4.7.3	3-Day Ahead Forecasting	53
4.8	Comparative Results	56
4.8.1	1-Day Ahead Comparison	56
4.8.2	3-Day Ahead Comparison	57
4.8.3	Discussion	59
5	Conclusion and Future Work	60
5.1	Conclusion	60
5.2	Future Work	61

Chapter 1

Introduction

1.1 Problem Statement and Motivation

The prediction of stock prices remains a persistent challenge in industry due to inherent market volatility, nonlinearity, and external influences [1]. Accurate predictions are essential for portfolio management, algorithmic trading, risk assessment, and financial planning [2]. While traditional ARIMA models assume linear dynamics and stationarity, the noisy and complex nature of financial markets often violates these assumptions [3].

The advancement of deep learning has led to models like LSTM and the Transformer, which are designed to learn long-range patterns in time series data [4]. At the same time, additive tools like Prophet are widely used for modeling seasonality and trend changes [5]. However, it remains unclear whether these new approaches consistently outperform well-tuned classical methods when forecasting stock prices [3].

1.2 Research Objectives and Questions

This study compares the performance of ARIMA, LSTM, Transformer, and Prophet models in predicting daily closing prices for Amazon (AMZN), Mi-

crosoft (MSFT), and ExxonMobil (XOM) stocks. These sectors represent diverse industries such as e-commerce, technology, and energy. Using Yahoo Finance data from January 1, 2022 to January 1, 2025, we evaluate 1 day and 3 day ahead forecasts within a walk-forward validation framework. Specifically, the research aims to:

- Evaluate the strengths and limitations of each model under consistent walk-forward validation settings.
- Compare the accuracy of the forecast quantitatively with MAE, MAPE, and RMSE.
- Examine how volatility, seasonality, and sector-specific factors affect model performance
- Provide practical guidance on the balances between classical statistical methods and deep-learning approaches for real-world stock forecasting

This study is guided by the following central questions:

- How do classical (ARIMA) and modern (LSTM, Transformer, Prophet) time series models compare in terms of out-of-sample forecasting performance on diverse stock market series?
- What are the implications of forecast horizon, model complexity, and data characteristics such as volatility, trend, and seasonality on prediction accuracy?
- Do neural architectures such as LSTM or Transformer offer practical benefits over statistical baselines for daily stock price prediction, or are their advantages mainly theoretical under conditions of limited data?

This comparative study aims to highlight the practical compromises in selecting financial forecasting models and to bridge the gap between classical statistics and modern data-driven methods for real-world stock prediction.

Chapter 2

Literature Review

Stock market forecasting is a long-standing and active area of research in financial engineering and data science. Traditional approaches utilize statistical models, while more recent approaches employ machine learning and deep learning-based models. This section focuses on ARIMA, LSTM, Transformer, and Prophet, which are representative models for time series prediction, and summarizes related literature.

2.1 Related Work

The ARIMA (Autoregressive Integrated Moving Average) model is a traditional time series forecasting method that performs well when the data show linear patterns and meets the assumption of stationarity. Recent studies in the financial field have demonstrated that ARIMA is an effective and practical tool for market forecasting. For example, this model effectively captures the patterns present in real market data, helping to predict future price movements [6]. Similarly, a comparative study by Ahammad et al. [7] examined the predictive accuracy of the ARIMA model on historical stock market data using standard metrics such as mean absolute error (MAE) and root mean square error (RMSE). The study reported a test MAE of 3.46 and a test

RMSE of 4.34, with ARIMA notably outperforming deep learning models, such as LSTM, in predictive precision.

However, ARIMA relies on stationarity and has limitations with nonlinear patterns, which can cause it to fall short in dynamic market conditions. This has led to growing interest in more advanced techniques such as Long Short-Term Memory (LSTM) networks [3]. LSTM network is a kind of recurrent neural network that helps find patterns and connections in sequences of data. LSTM works well for financial time series forecasting because financial data often exhibits high volatility, non-stationarity, and is subject to change over time. Unlike traditional models, LSTM can retain important information for longer, which is useful for predicting tasks such as stock prices [8]. Recent studies show that LSTM models are reliable and useful for forecasting financial markets. For instance, Mroua et al. [9] utilized LSTM to predict several financial indices, including the S&P GSCI and other major stock market benchmarks, during the COVID-19 pandemic. The model achieved very low MAE and RMSE values, which highlights the effectiveness of LSTMs in capturing nonlinear trends and making accurate predictions. Similarly, Pragnya et al. [10] also tested the performance of LSTMs using historical stock data, including closing and opening prices, highs, lows, and trading volume. Their study found that LSTM-based models regularly had lower mean absolute error (MAE) and root mean square error (RMSE) scores than traditional statistical methods when working with complex real-world financial data. A study by Osman [11] evaluated LSTM, ARIMA, and VAR models on S&P 500 daily closing prices from 2015 to 2020, a period of high market volatility. LSTM had a significantly lower RMSE (43.25) than ARIMA (92.69), demonstrating consistent performance and accuracy even in volatile market environments.

Although Long Short-Term Memory (LSTM) networks demonstrate notable strengths, their performance often declines when processing extended sequences or sparse datasets. Transformer models utilize self-attention mech-

anisms to effectively capture long-range dependencies and have become a robust alternative. In contrast to LSTM networks, Transformer architectures operate without sequential processing, which enables more efficient parallel computation and improved scalability. Additionally, they have performed well in predicting financial time series, which often show complex patterns, high volatility, and nonlinear relationships over long periods [4]. Recent studies have emphasized the robustness and superior performance of transformer models in financial market forecasting. For example, Nguyen et al. [12] used a transformer-based architecture to predict stock prices using multivariate time series data from major indices such as the S&P 500 and NASDAQ. Despite complex market volatility, this model achieved low error rates and effectively identified market trends. Similarly, Wang et al. [13] evaluated Transformers on historical stock data. Their results showed that Transformer models consistently achieved lower MAE and RMSE compared to LSTM and ARIMA. This highlights that Transformers demonstrate accurate predictions on non-stationary and noisy financial datasets. Zhang et al. [14] compared Transformer models with LSTM and other neural networks using daily closing prices from the Shanghai Stock Exchange between 2016 and 2022, including times of economic uncertainty. The Transformer achieved an RMSE of 1.634, outperforming LSTM's 1.745. This result highlights the Transformer's strength in modeling long-term patterns and making reliable predictions during volatile periods.

While neural models such as LSTM and Transformer offer strong performance, they can be hard to interpret and often need a lot of data and tuning. In contrast, Facebook's Prophet model is easier to interpret and flexible. It uses regression to capture trends, seasonality, and holiday effects. It works well for financial forecasting, where market data can show nonlinear trends, periodic fluctuations, and external shocks. Prophet stands out from traditional models because it easily adapts to changes and seasonal shifts, making it practical for predicting variables such as stock prices and

exchange rates, even when markets are unstable [5]. Recent research highlights the Prophet model as a strong and practical tool for predicting trends in financial markets. For example, Kolari et al. [5] applied Prophet to predict bank capital ratios by combining bank financial ratios and macroeconomic data for large US banks from 2005 to 2020. The model achieved lower forecasting errors than the benchmark ARIMAX models. The model accurately captured trends even during times of economic stress, as demonstrated by both in-sample and out-of-sample results. Similarly, Kakinuma [15] evaluated Prophet alongside LSTM and ARIMA to predict the prices of financial innovation funds. Using MAE, MAPE, and RMSE as evaluation metrics, the results indicated that Prophet outperformed LSTM on smaller datasets due to its ability to handle seasonality and outliers effectively. Hakkar et al. [16] compared Prophet, ARIMA, and LSTM to predict the GBP/USD exchange rate using daily data from 2023 to 2025. Prophet achieved low MAE, RMSE, and MAPE values, showing it can be effective in volatile foreign exchange markets.

2.2 Summary

In summary, ARIMA is a reliable basic model for linear data and static conditions. On the other hand, LSTM and Transformer models demonstrate excellent performance in complex and nonlinear time series modeling. Prophet shows outstanding flexibility and interpretability, especially in situations with strong seasonality and limited data. These results emphasize the importance of selecting a prediction model based on data characteristics and specific market conditions.

Chapter 3

Methodology

3.1 Data Preprocessing

Preprocessing plays an important role in getting financial time series data ready for model training and evaluation. The raw stock price data from Yahoo Finance is cleaned and transformed through several steps to improve model performance and make the data more reliable.

- **Handling Missing Data:** Financial data sets often contain missing values due to market disruptions or recording errors. If these gaps are not addressed, predictions can become inaccurate. Using methods such as model-based or multiple imputation helps maintain the reliability of the data and reduces bias [17].
- **Feature Selection:** Financial datasets often include numerous features, but only a subset is helpful for predicting stock prices. Feature selection helps improve model performance. Methods such as random forest and principal component analysis were used to identify the most important variables, which reduces noise, increases efficiency, and makes the results easier to interpret [18].

- **Outlier Handling:** Outliers in financial time series can lead to bias in model training and reduce forecast accuracy. To manage this, statistical methods and visual tools, such as box plots, help identify outliers, which can then be winsorized or removed [19].

3.2 Model Selection

This study employed four prediction models, each selected for its unique strengths. ARIMA represents traditional linear methods and works well with stable data. LSTM was selected for its ability to handle sequential patterns in volatile time series. The Transformer model helps manage long-term dependencies using its attention mechanism. Prophet was included because it offers an interpretable framework that effectively models seasonality, trends, and holiday effects.

3.2.1 ARIMA

The Autoregressive Integrated Moving Average (ARIMA) model is a widely used method for time series forecasting, particularly in financial data with linear patterns. For ARIMA to work properly, the time series must be stationary. This means the data's mean, variance, and autocorrelation stay consistent over time. [20]

Formally, a stochastic process $\{X_t\}$ is strictly stationary if its joint distribution does not change when shifted in time. That is, for any $n \in \mathbb{N}$ and shift $\tau \in \mathbb{Z}$: [20]

$$P(X_{t_1}, X_{t_2}, \dots, X_{t_n}) = P(X_{t_1+\tau}, X_{t_2+\tau}, \dots, X_{t_n+\tau}). \quad (3.1)$$

ARIMA builds on autoregressive (AR) and moving average (MA) methods, adding differencing to manage non-stationary data. This makes it useful for analyzing economic and financial trends. The general formulation of the

ARIMA model can be expressed using the backshift (lag) operator as follows: [20]

$$(1 - \sum_{i=1}^p \phi_i L^i) (1 - L)^d y_t = (1 + \sum_{j=1}^q \theta_j L^j) \epsilon_t \quad (3.2)$$

Here, y_t is the value of the series at time t , L the lag operator, ϕ_i the AR parameters, d the differencing order needed for stationarity, θ_j the MA parameters, and ϵ_t white noise.

ARIMA decomposes a time series into components such as trend and noise. It uses differencing and autoregressive terms to model trends, while moving average terms capture short-term irregularities. This decomposition assumes that future behavior will be similar to past trends, helping the model filter out random fluctuations and emphasize underlying patterns [20].

To determine the differencing order d , the Augmented Dickey-Fuller (ADF) test is applied. This test evaluates the null hypothesis that a unit root exists, indicating non-stationarity in the series. The test statistic is computed as follows: [21]

$$t = \frac{\hat{\gamma}}{\text{SE}(\hat{\gamma})} \quad (3.3)$$

where $\hat{\gamma}$ is the estimated coefficient in the regression

$$\Delta y_t = \gamma y_{t-1} + \sum_{i=1}^p \beta_i \Delta y_{t-i} + \epsilon_t, \quad (3.4)$$

and $\text{SE}(\hat{\gamma})$ is its standard error. If the p-value exceeds 0.05, $d = 1$ is applied. Model orders p and q are selected by minimizing the Akaike Information Criterion (AIC):

$$\text{AIC} = 2k - 2 \ln(\hat{L}), \quad (3.5)$$

where k is the number of parameters and $\hat{L}$ is the maximized likelihood of the model. [21]

To apply ARIMA in practice, one typically follows these steps, as adapted from [3], [21]:

1. Split the data into training, validation, and test sets. In this study, 80% of the data was used for training, 10% for validation, and 10% for testing. In the training dataset, a working forward validation method was applied, in which the prediction window was sequentially updated with new test values. This method does not train the model only once, but rather retrains it whenever new data is added, thereby continuously improving the model.
2. Check for stationarity using the augmented Dickey-Fuller (ADF) test. If the p-value was greater than 0.05, the series was differenced once ($d = 1$) to achieve stationarity.
3. Select optimal model order using grid search and AIC. A grid search over AR (p) and MA (q) terms in the range $\{0, 1, 2, 3\}$ was performed while keeping d fixed. The model with the lowest Akaike Information Criterion (AIC) was selected to balance accuracy and complexity.
4. Fit and retrain the model using a rolling window. In this study, when the training data set exceeds a predefined length, only the most recent 200 observations were used to fit the initial model. During walk-forward forecasting, the model was retrained with the updated window at each step to forecast the next value.
5. Visualize the forecasts. The predicted values were plotted alongside the actual values from the training, validation, and test sets to evaluate how well the model captured the behavior of time series.
6. Evaluate the accuracy of forecast. The prediction performance is measured by comparing the predicted values with the actual test values using the mean absolute error (MAE), root mean square error (RMSE), and mean absolute percentage error (MAPE).

3.2.2 LSTM

The Long Short-Term Memory (LSTM) network is a type of recurrent neural network (RNN) designed to address the vanishing gradient problem, where gradients shrink during backpropagation and hinder the learning of long-term dependencies in standard RNNs. This makes LSTM effective for modeling long-range relationships in sequential data, such as financial time series [22]. LSTM works especially well for financial forecasting, where data can be non-linear, volatile, and show patterns such as trends or seasonality [8].

Mathematically, an LSTM cell at time step t is defined by the following gates [22]:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (3.6)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad (3.7)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C), \quad (3.8)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (3.9)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad (3.10)$$

$$h_t = o_t \odot \tanh(C_t), \quad (3.11)$$

where x_t is the input, h_{t-1} the previous hidden state, C_{t-1} the previous cell state, W_* and b_* are weights and biases, σ is the sigmoid activation, $\tanh$ the hyperbolic tangent, and $\odot$ element-wise multiplication.

LSTM is trained via Backpropagation Through Time (BPTT), minimizing a loss function such as [22]:

$$\mathcal{L} = \sum (y_t - \hat{y}_t)^2. \quad (3.12)$$

For a sequence of length T , the gradient of the loss with respect to weights W is [22]:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W}, \quad (3.13)$$

where $\mathcal{L}_t$ is the loss at time t , and gradients are propagated backward to update parameters while mitigating vanishing gradients through the cell state.

The cell state C_t stores important past information while the gates selectively update or forget inputs. This helps the model focus on meaningful temporal relationships. This structure enables LSTMs to learn underlying patterns and generalize them for forecasting as long as the main temporal dynamics remain stable [8].

To apply LSTM in practice, one typically follows these steps, as adapted from [23]:

1. Normalize the data and form input sequences using a sliding window approach. In this study, MinMax scaling - a normalization technique that rescales features to the range $[0, 1]$ - was applied to the closing prices. A lookback window of 30 days was selected to balance the sequence length and computation time.
2. Split the dataset into training, validation, and test sets to prevent overfitting. Here, the data was divided into 80% for training, 10% for validation, and 10% for testing, with the training set used in a rolling-window approach to perform walk-forward validation. In this approach, a model is trained using a fixed-size window composed of the most recent observations, and as new data points are added, the window is shifted forward by one step, continuously updating the model. This process optimizes the model by simulating real-time predictions.
3. Set up the model by choosing the number of LSTM layers, hidden units, and training settings such as learning rate and batch size. In this study, a single LSTM layer with 64 hidden units and L2 regularization, which penalizes large weights to prevent overfitting was used. Batch

normalization was applied to stabilize training by standardizing the inputs to each layer, and dropout was used to randomly deactivate neurons during training, which helps reduce overfitting. The model was trained with a learning rate of 0.001 and a batch size of 32.

4. Train the model using backpropagation over time, with a loss function such as mean squared error (MSE) and an optimizer such as Adam. In this study, the model was trained for up to 30 epochs using MAE as the loss function and the Adam optimizer. Early stopping was used to avoid overfitting by halting training when the validation loss stopped improving.
5. Evaluate on unseen test data, applying dropout and adaptive learning rate scheduling, which adjusts the learning rate based on performance, to improve generalization.
6. Evaluate final performance with metrics such as MAE, RMSE, and MAPE.

3.2.3 Transformer

The Transformer model is a type of neural network that uses attention mechanisms instead of recurrence to understand relationships in data [24]. Attention allows the model to weigh the importance of each input element dynamically, enabling it to capture both short and long-term dependencies [24]. This approach works well for financial time series forecasting, where it can handle long-term patterns, non-linearity, and volatility, making it suitable for tasks like predicting stock prices [25].

Mathematically, the core of the Transformer is the self-attention mechanism, which computes the relationships between all elements in a sequence [24]. For a single attention head, the queries (Q), keys (K), and values (V) are computed as:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (3.14)$$

where X is the input sequence, and W_Q , W_K , and W_V are learnable projection matrices.

The scaled dot-product attention is then calculated as [24]:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V, \quad (3.15)$$

where d_k is the dimension of the keys and the softmax function normalizes the attention scores to form a probability distribution.

To allow the model to jointly attend to information from different representation subspaces, multi-head attention uses multiple attention heads and concatenates their outputs [24]:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \quad (3.16)$$

where each attention head is defined as:

$$\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i}), \quad (3.17)$$

and W_{Q_i} , W_{K_i} , W_{V_i} , and W_O are projection matrices learned during training.

The full Transformer encoder stacks these with position-wise feed-forward networks, layer normalization, and positional encodings to incorporate sequence order [24]:

$$PE_{pos,2i} = \sin \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right), \quad PE_{pos,2i+1} = \cos \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right) \quad (3.18)$$

The Transformer model identifies patterns in time series data, such as trends, seasonality, and noise, by learning contextual relationships through its attention layers. Multi-head attention identifies long-term trends and re-

curing patterns by focusing on distant points in the data. Feed-forward layers reduce noise and highlight local changes. The goal is to learn contextual representations, where trends indicate overall direction, seasonality captures periodic cycles, and noise is minimized to improve forecasting. [25, 26]

To apply Transformer in practice, one typically follows these steps, as adapted from [12]:

1. Normalize the data and add positional encodings, then create input sequences with optional masking for decoder. Masking in the decoder ensures that future time steps are not used to predict current values, preserving the causality of forecasting. In this study, MinMax scaling was applied to the closing prices, positional encodings were added using sine-cosine functions to retain the order of the sequence, since the Transformer does not inherently process sequence order. Input sequences were created with a 60-day lookback to balance historical context and computational efficiency.
2. Divide the dataset into training, validation, and test sets to mitigate overfitting. Here, an 80/10/10 split was employed, and training data was used in a rolling-window walk-forward validation to help prevent overfitting.
3. Specify the model's structure such as encoder/decoder layers, attention heads, embedding size and training parameters like learning rate, batch size. In this study, the model used two transformer blocks with four attention heads, a key dimension of 32, and 64 feed-forward units. Training parameters included a learning rate of 0.0005 and a batch size of 32.
4. Train using backpropagation with a loss like mean squared error (MSE) and an optimizer such as AdamW. In this study, training used Huber loss, which combines the benefits of MSE and MAE to reduce the impact of outliers, and the AdamW optimizer, which decouples weight

decay from gradient updates for improved regularization. The model was trained for up to 25 epochs with early stopping.

5. Apply regularization techniques such as dropout within attention layers to handle noise, and validation metrics like MAE or RMSE. Here, dropout at a rate of 0.1 was used in attention and feed-forward layers to reduce noise. Validation relied on MAE, MAPE, and RMSE to check generalization.
6. Assess performance and refine, often through hybridization with specialized adaptations like Temporal Fusion Transformers for time series. In this study, performance was assessed using inverse-transformed predictions. Fine-tuning included weight decay of 0.01, a regularization technique that penalizes large weights to prevent overfitting, and early stopping based on patience. No hybrid models were used to keep results comparable to the baseline.

3.2.4 Prophet

The Prophet model is an additive regression framework developed by Facebook (now Meta) to forecast time series data that exhibit strong seasonal patterns, trends, and effects from events such as holidays. It works well for financial time series, which often exhibit nonlinear, volatile, and periodic changes, and it provides clear and scalable forecasts with minimal manual adjustment. [27]

The Prophet model decomposes the time series into an additive form [27]:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t, \quad (3.19)$$

where

- $y(t)$ is the observed value at time t ,

- $g(t)$ represents the non-periodic trend, such as piecewise linear or logistic growth,
- $s(t)$ describes periodic seasonality, for example, yearly or weekly cycles using a Fourier series,
- $h(t)$ reflects the effects of holidays or events, modeled with binary indicators that can be manually specified or estimated from data,
- ϵ_t is the error term, assumed to follow a normal distribution with possible changes in variance.

In this study, the model used log-transformed closing prices to minimize volatility and ensure predictions remain positive. The model was also adjusted to better capture the unique trends and seasonal changes of each stock. Prophet uses changepoints to account for shifts in these trends. Changepoints are specific time points at which the trend can shift due to market regime changes or sudden external events. Prophet automatically detects these points, allowing for flexible modeling of piecewise linear trends [27].

The trend component $g(t)$ is defined with changepoints for flexibility [27]:

$$g(t) = (k + \mathbf{a}(t)^T \delta) t + (m + \mathbf{a}(t)^T \gamma), \quad (3.20)$$

where

- k is the base growth rate that governs the initial slope of the trend,
- m is the offset that shifts the curve vertically,
- δ represents rate adjustments at each changepoint,
- γ ensures continuity so the trend curve remains smooth even after the changepoint,
- $\mathbf{a}(t)$ is the changepoint indicator vector.

For this analysis, linear growth was assumed with automatic changepoint detection, adjusted via a prior scale of 0.5 to balance flexibility and overfitting.

Seasonality $s(t)$ is modeled via a Fourier series [27]:

$$s(t) = \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right), \quad (3.21)$$

with period P and coefficients a_n, b_n learned from the data. In this study, custom monthly (period 30.5, Fourier order 5) and quarterly (period 91.25, Fourier order 3) seasonalities were added. Weekly was enabled and yearly/daily was disabled to align with stock market patterns.

Prophet separates time series data into trend, seasonality, and noise, which helps improve forecasting accuracy. The trend is identified using changepoints, seasonality is captured by Fourier terms, and noise is handled by the error term, which also addresses outliers and missing data. This approach assumes that future patterns will be similar to past ones, but it can still accommodate unexpected changes. [27]

To apply Prophet [5]:

1. Prepare your data in a data frame with columns `datetime` and `y` (value); optionally add regressors or holiday indicators. In this study, daily closing prices were formatted in this manner, and US holidays, such as Christmas and Independence Day, were included for the years 2022 to 2025 to account for market closures.
2. Fit the model using Stan, a probabilistic programming engine for Bayesian inference, which estimates the trend, seasonality, and holiday effects [15]. In this study, the model was fitted using rolling windows of 200 days, with a changepoint prior scale of 0.5 and seasonality prior scale of 10.0 to enhance adaptability.
3. Create a future data frame for the forecast horizon and call `predict`

function to generate forecasts. For this analysis, 1- and 3-day horizons were predicted on log-transformed data, with predictions exponentiated back to the original scale.

4. Evaluate with cross-validation metrics such as MAE, RMSE and adjust by adding custom seasonality or regressors like macro indicators for volatility [15]. In this study, MAE, MAPE, and RMSE were computed on the test data, with custom seasonalities added to enhance the handling of financial cycles, although no external regressors were used to maintain focus on univariate forecasting.

3.3 Model Evaluation Metrics

Three standard metrics were used to evaluate the forecasting accuracy of the ARIMA, LSTM, Transformer, and Prophet models in predicting stock prices: Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and Root Mean Squared Error (RMSE) [28]. These metrics provide different insights into the performance of the model. MAE quantifies average absolute deviations, MAPE expresses errors as scale-independent percentages, and RMSE assigns greater weight to larger discrepancies. These metrics are widely adopted in financial time series analysis due to their effectiveness in quantifying prediction errors in volatile and non-linear data, which facilitates fair comparisons between forecasting methods [28].

Mean Absolute Error (MAE)

MAE measures the average magnitude of the forecasting errors, irrespective of direction [29]:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (3.22)$$

where y_i is the actual value, $\hat{y}_i$ the predicted value, and n the number of observations. MAE is intuitive and not easily influenced by outliers, so it works well in financial forecasting, especially when absolute errors can impact decisions like estimating average mispricing in stock predictions [29].

Mean Absolute Percentage Error (MAPE)

MAPE expresses the average error as a percentage of the actual values [30]:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%, \quad (3.23)$$

which is scale-invariant and useful for comparing performance across different price ranges or volatile markets. However, MAPE can be sensitive when actual values y_i are near zero [30].

Root Mean Squared Error (RMSE)

RMSE quantifies the square root of the average squared errors, emphasizing larger deviations [29]:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3.24)$$

RMSE calculates the average of squared errors, giving more weight to large mistakes than MAE does. This is useful in finance because big prediction errors can have serious effects, such as during market crashes. Since RMSE uses the same units as the target, it is also easier to understand how accurate a forecast is [29].

These metrics collectively assess how well each model captures trends, seasonality, and noise in stock price data [28]. Lower values indicate better performance, with RMSE often prioritized in volatile scenarios to account

for high-impact errors. In this study, they were computed on out-of-sample test data to ensure unbiased evaluation [\[28\]](#).

Chapter 4

Results and Discussion

4.1 Dataset

Historical stock price data from Yahoo Finance is utilized to evaluate the performance of various forecasting models. Three major publicly traded companies are selected to represent diverse market sectors: Amazon (AMZN) in consumer discretionary, Microsoft (MSFT) in technology, and ExxonMobil (XOM) in energy.

The dataset comprises three years of data, spanning from January 1, 2022 to January 1, 2025. The yfinance application programming interface (API) supplies multiple price attributes, such as open, high, low, and close. For this analysis, only the daily closing price for each ticker is retained. Restricting the input features and target variable to the closing price streamlines the modeling process and preserves the most relevant price information.

4.2 Data Preprocessing

The dataset is split into training, validation, and testing sets, with 80% used for training, and 10% each for validation and testing. Since there are no duplicates or missing values, no extra data preprocessing is needed before

training the model. Outliers are found using the interquartile range (IQR) method and checked with box plots. As illustrated in Figure 4.1, no significant outliers are observed in AMZN or MSFT. However, several low-end outliers appear in XOM, represented by the data points extending beyond the lower whisker of the box plot. In a box plot, the box spans from the first quartile (Q_1) to the third quartile (Q_3), with the line inside the box indicating the median (second quartile). The whiskers represent the range of data within 1.5 times the interquartile range (IQR) between the lower and upper quartiles, indicating that most of the data is within the normal range. The whiskers extend to the most extreme non-outlier values within the range $[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}]$, and values outside this range are marked as outliers. The whiskers can appear asymmetric when the distribution of the data is skewed, resulting in a longer tail on one side. Winsorization was applied to reduce the influence of these extreme values while maintaining the basic structure of the data. This method replaces values outside the IQR range with a threshold value rather than removing them. As a result, the data is minimally distorted by outliers, maintaining its original form and becoming a more suitable dataset for predictive modeling.

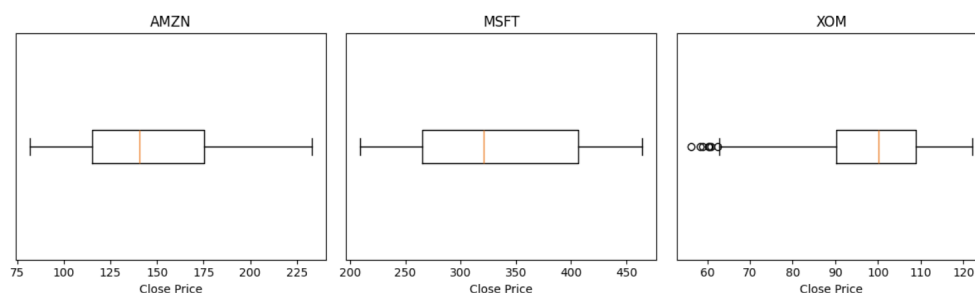


Figure 4.1: Boxplots of daily closing prices for AMZN, MSFT, and XOM

4.3 Exploratory Data Analysis

To provide an initial analysis of stock price movements, the daily closing prices of Amazon (AMZN), Microsoft (MSFT), and ExxonMobil (XOM) are plotted for the period from January 1, 2022 to January 1, 2025 (see Figure 4.2). The resulting plot illustrates the trends and volatility associated with each stock.

The figure demonstrates that each stock exhibits distinct temporal behavior. Microsoft (MSFT) displays a steady upward trend beginning in mid-2022. Amazon (AMZN) exhibits greater volatility but also a gradual increase in price. ExxonMobil (XOM) presents a modest upward trend with comparatively low volatility. These observed patterns underscore the significance of sector diversity in financial time series forecasting.



Figure 4.2: Daily closing prices for AMZN, MSFT, and XOM

4.4 ARIMA Model Results

4.4.1 Model Fitting Procedure

For each ticker symbol, the optimal order of the autoregressive integrated moving average (ARIMA) model was determined in several steps. First, the appropriate level of differencing d was identified using the augmented Dickey–Fuller (ADF) test on the training dataset. If the ADF p-value was greater than 0.05, indicating non-stationarity, d was set to 1. Otherwise, d was set to 0. Subsequently, the autoregressive order p and the moving average order q were selected by performing a grid search to minimize the Akaike Information Criterion (AIC) among candidate models with p and q values up to 3. The final ARIMA model for each ticker symbol was tuned using only the training data.

4.4.2 1-Day Ahead Forecasting

The 1-day ahead forecasting performance of the ARIMA model was evaluated for Amazon (AMZN), Microsoft (MSFT), and ExxonMobil (XOM) using a rolling-window approach with a length of 200 observations. Table 4.1 summarizes the selected ARIMA model orders and test results for each ticker, while Figure 4.3 presents the corresponding forecast plots.

Ticker	ADF p-value	Selected ARIMA Order	AIC
AMZN	0.662	(3, 1, 3)	3108.2
MSFT	0.972	(2, 1, 3)	3687.1
XOM	0.171	(2, 1, 2)	2289.3

Table 4.1: ADF test results and selected ARIMA orders for each ticker

The ADF p-values for all three tickers exceeded 0.05, indicating that the closing price series are non-stationary and require first-order differencing

($d = 1$). A grid search for the AR and MA orders (p, q) , based on the minimum AIC, resulted in moderately complex models for each stocks.

Figure 4.3 presents the actual and predicted closing prices for each ticker in the test set, using the 1-step-ahead ARIMA forecasts.

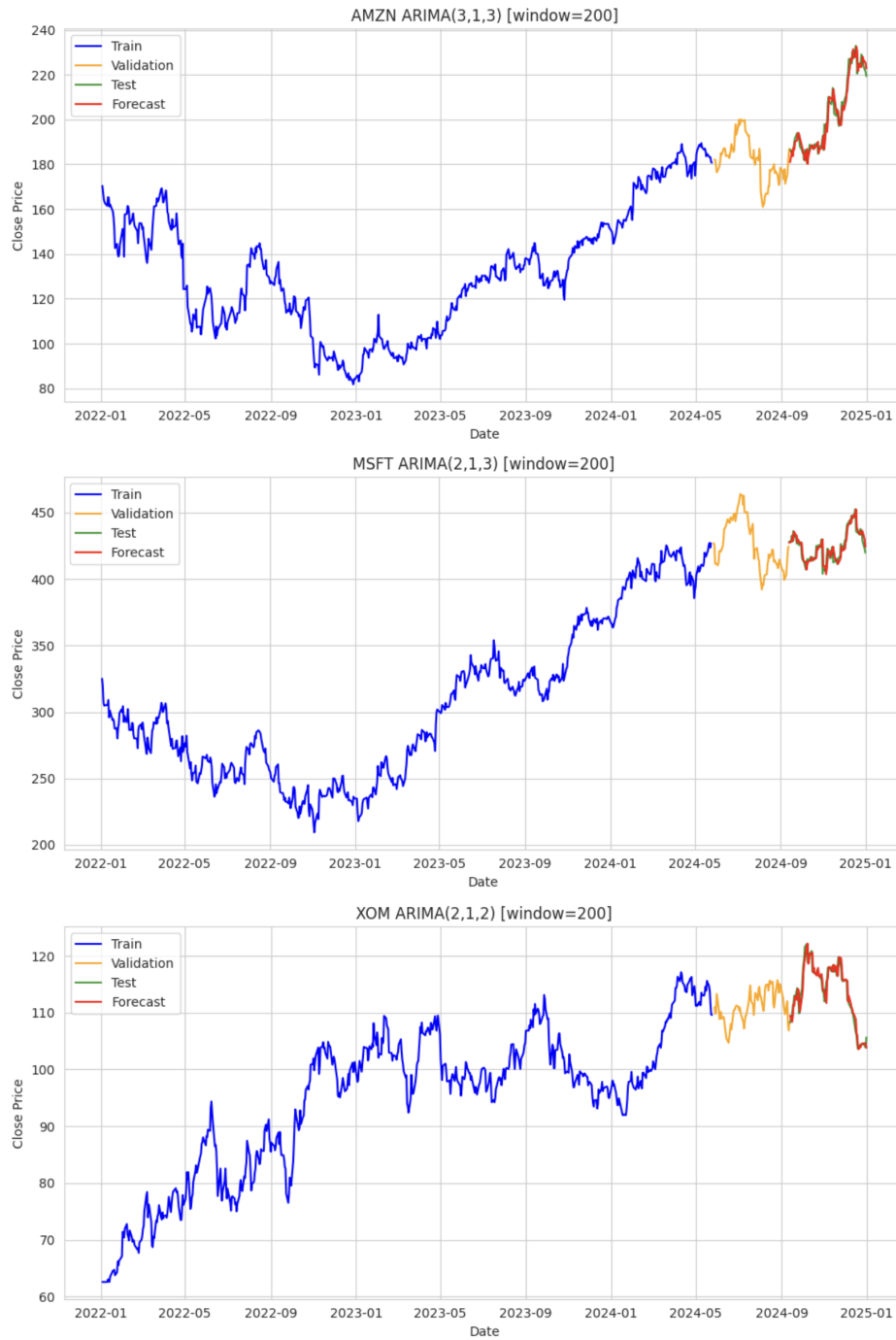


Figure 4.3: ARIMA 1-day ahead forecasts (Train, Validation, Test, Forecast) for (top) AMZN, (middle) MSFT, and (bottom) XOM.

The forecast curve (red) generally follows the short-term trend of the actual stock price (green), but performance varies slightly depending on the stock.

Ticker	MAE	MAPE (%)	RMSE
AMZN	2.87	1.41	3.66
MSFT	3.75	0.88	5.46
XOM	1.03	0.90	1.37

Table 4.2: Test set performance of ARIMA 1-day forecasts

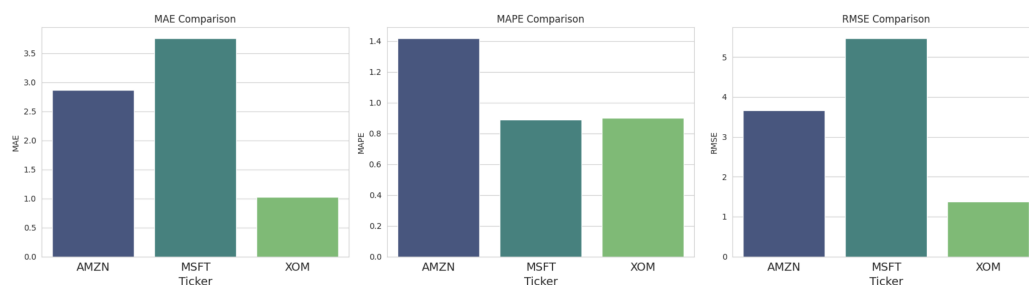


Figure 4.4: Comparison of ARIMA 1-day forecast performance for each ticker: (left) MAE, (center) MAPE, (right) RMSE.

As summarized in Table 4.2 and Figure 4.4, key findings are:

- **AMZN (Amazon):** MAE 2.87, MAPE 1.41%, RMSE 3.66. These error values are considered moderate and are attributed to the higher price volatility observed in Amazon stock. The slightly elevated RMSE indicates the presence of occasional larger deviations in the predictions.
- **MSFT (Microsoft):** MAE 3.75, MAPE 0.88%, RMSE 5.46. These results indicate relatively accurate proportional forecasts but greater absolute errors. The elevated errors may be attributable to fluctuations within the technology sector.

- **XOM (ExxonMobil):** MAE 1.03, MAPE 0.90%, RMSE 1.37. These results demonstrate superior predictive accuracy within the energy sector, where price movements are relatively stable.

4.4.3 3-Day Ahead Forecasting

The 3-day ahead block forecasting was performed using ARIMA models for Amazon (AMZN), Microsoft (MSFT), and ExxonMobil (XOM). The model order selection process mirrored the 1-day forecasting approach, utilizing ADF test for differencing and an AIC-based grid search. The primary distinction is the walk-forward mechanism. The model is trained on the initial historical data, forecasts a 3-day block using the final day's value as the prediction, updates the training history with the actual three-day values, and then proceeds to the subsequent block. The selected model orders are consistent with those reported in Table 4.1, and Figure 4.5 shows the forecast plots.



Figure 4.5: ARIMA 3-day ahead block forecasts (Train, Validation, Test, Forecast) for (top) AMZN, (middle) MSFT, and (bottom) XOM.

The forecast points (red) generally followed the direction trends of the actual stock prices (green). However, the discrepancy between forecasted and actual values increased compared to the 1-day forecasts, particularly during periods of rapid price fluctuation. Performance also varied by ticker symbol, indicating that sector characteristics and extended prediction horizons influence forecasting accuracy.

Ticker	MAE	MAPE (%)	RMSE
AMZN	4.88	2.37	5.79
MSFT	6.39	1.51	8.37
XOM	2.16	1.90	2.67

Table 4.3: Test set performance of ARIMA 3-day forecasts

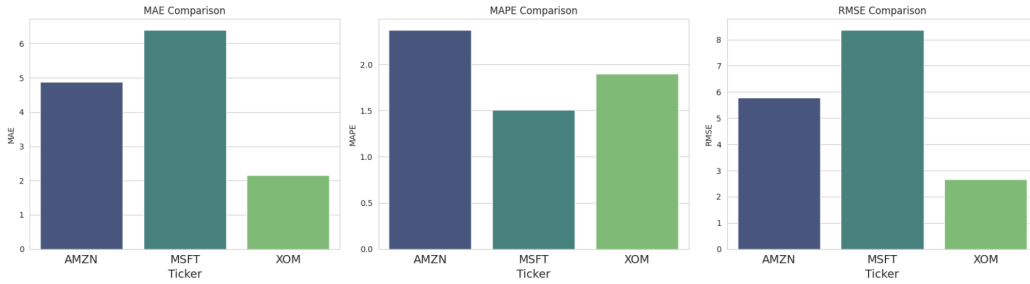


Figure 4.6: Comparison of ARIMA 3-day forecast performance for each ticker: (left) MAE, (center) MAPE, (right) RMSE.

As summarized in Table 4.3 and Figure 4.6, key findings are:

- **AMZN (Amazon):** MAE 4.88, MAPE 2.37%, RMSE 5.79. The increased error values relative to shorter time frames indicate that the volatility of Amazon's stock complicates accurate multi-day forecasting.
- **MSFT (Microsoft):** MAE 6.39, MAPE 1.51%, RMSE 8.37. Despite relatively acceptable percentage errors, the magnitude of absolute errors suggests significant challenges in forecasting extended trends within the technology sector.

- **XOM (ExxonMobil):** MAE 2.16, MAPE 1.90%, RMSE 2.67. These results may reflect the relatively stable price patterns characteristic of the energy sector, although error metrics increased modestly over the extended forecast period.

When comparing the 1-day and 3-day results, the 3-day forecasts had higher errors for all metrics and tickers. On average, MAE increased by about 83%, MAPE by 84%, and RMSE by 69%. This drop in accuracy is likely due to growing uncertainty over longer time frames, since ARIMA’s linear approach can make errors worse in volatile stocks like AMZN and MSFT. However, XOM’s steadier performance led to smaller increases in error, which suggests that ARIMA works better for stable assets and is less reliable for multi-day forecasts in more unpredictable markets.

4.5 LSTM Model Results

4.5.1 Model Fitting Procedure

A Long Short-Term Memory (LSTM) neural network was implemented to assess the performance of deep learning models in predicting stock prices one day and three days ahead. The closing price data for each stock was divided into a training set (80%), a validation set (10%), and a test set (10%). A lookback window of 30 days was applied to enhance sequence learning. A rolling window of 200 data points was used for forward predictions. The LSTM model was retrained at each prediction step using the most recent data within the rolling window.

The LSTM architecture comprised a single LSTM layer (64 units) with L2 regularization, followed by batch normalization, dropout, and a dense output layer. MAE was used as the loss function, and Adam as the optimizer. Training was conducted for up to 30 epochs with early stopping and adaptive learning rate scheduling.

All price series were MinMax-scaled based on the training and validation data. Predictions for each day or 3-day block were recursively generated, and all evaluation metrics were computed on the original unscaled test series.

4.5.2 1-Day Ahead Forecasting

Table 4.4 summarizes LSTM performance for the 1-day ahead forecasting task, while Figure 4.7 illustrates the actual and predicted trajectories.

Ticker	MAE	MAPE (%)	RMSE
AMZN	14.10	6.67	17.38
MSFT	9.71	2.25	12.40
XOM	4.10	3.68	5.22

Table 4.4: Test set performance of LSTM 1-day forecasts

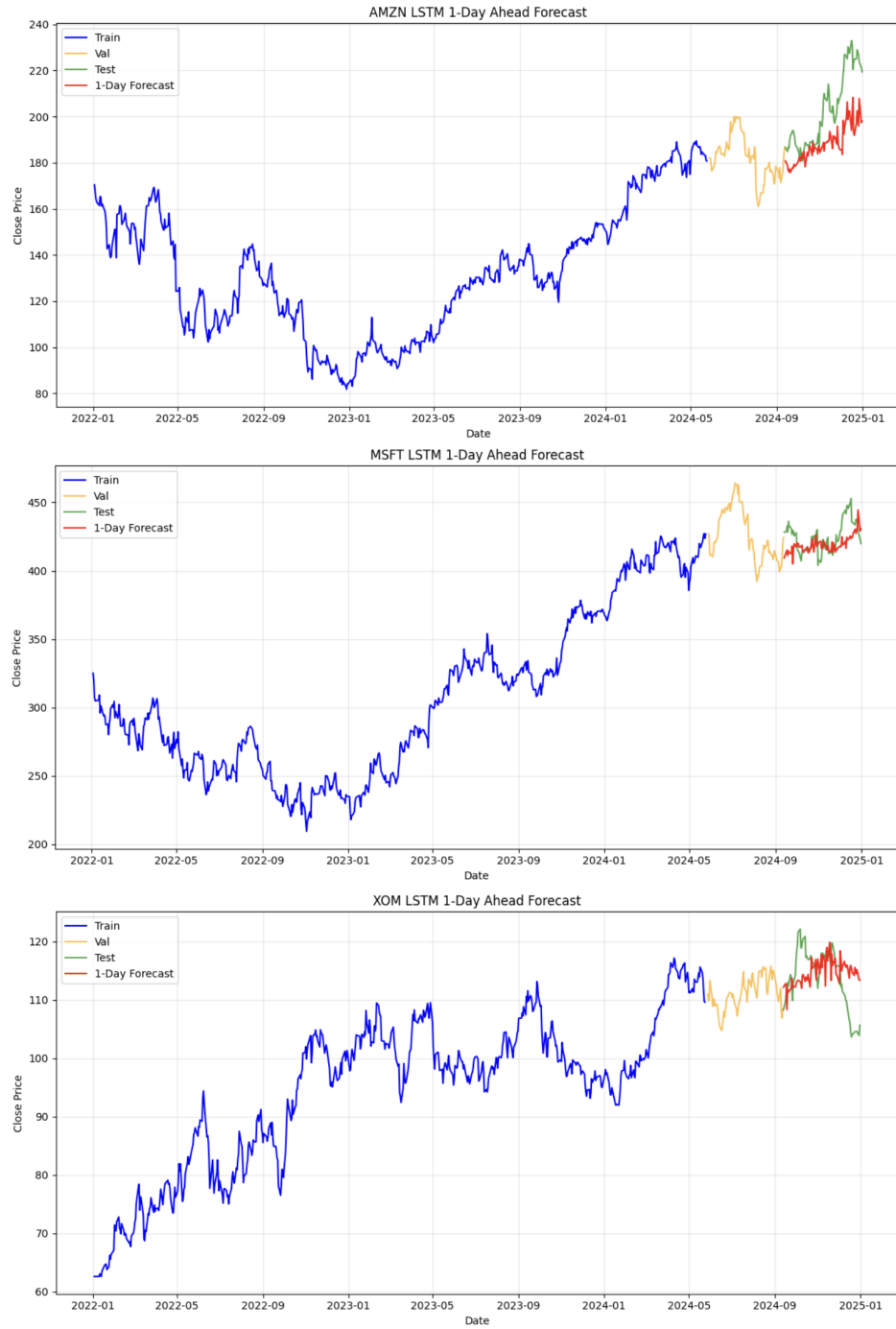


Figure 4.7: LSTM 1-day ahead forecasts (Train, Validation, Test, Forecast) for (top) AMZN, (middle) MSFT, and (bottom) XOM.

As seen in Table 4.4 and Figure 4.7, the LSTM predictions often lagged behind actual price trends, particularly during rapid swings. Errors were notably larger than for ARIMA or Prophet across all tickers:

- **AMZN (Amazon):** MAE 14.10, MAPE 6.67%, RMSE 17.38. These values represent the highest errors among all models, indicating that the Long Short-Term Memory (LSTM) model demonstrates limited adaptability to high volatility in shorter time series.
- **MSFT (Microsoft):** MAE 9.71, MAPE 2.25%, RMSE 12.40. The model produced a moderate proportional error and a relatively large absolute error, particularly during periods of increased volatility.
- **XOM (ExxonMobil):** MAE 4.10, MAPE 3.68%, RMSE 5.22. Although the absolute error was lower than that observed for technology stocks, the LSTM model continued to underperform relative to statistical models.

4.5.3 3-Day Ahead Forecasting

The same LSTM architecture and walk-forward protocol were applied for 3-day ahead block forecasts: the model was trained on each rolling window, predicted three consecutive days, and used the last of those as the representative forecast, before moving the window forward.

Table 4.5 presents the performance metrics for the LSTM 3-day forecasting task. Figure 4.8 illustrates the comparison between predicted and actual time series values.

Ticker	MAE	MAPE (%)	RMSE
AMZN	15.07	7.09	18.70
MSFT	9.73	2.25	12.99
XOM	4.56	4.09	5.59

Table 4.5: Test set performance of LSTM 3-day forecasts



Figure 4.8: LSTM 3-day ahead block forecasts (Train, Validation, Test, Forecast) for (top) AMZN, (middle) MSFT, and (bottom) XOM.

Errors consistently increased for the longer forecast horizon, as summarized in Table 4.5 and Figure 4.9:

- **AMZN (Amazon):** MAE 15.07, MAPE 7.09%, RMSE 18.70 The model’s performance declined for longer horizons, likely as a result of increased forecast uncertainty.
- **MSFT (Microsoft):** MAE 9.73, MAPE 2.25%, RMSE 12.99. These results are similar to the 1-day forecast, with a marginal increase in RMSE.
- **XOM (ExxonMobil):** MAE 4.56, MAPE 4.09%, RMSE 5.59. This represents the strongest LSTM performance among the three stocks, although it remains inferior to classical models.

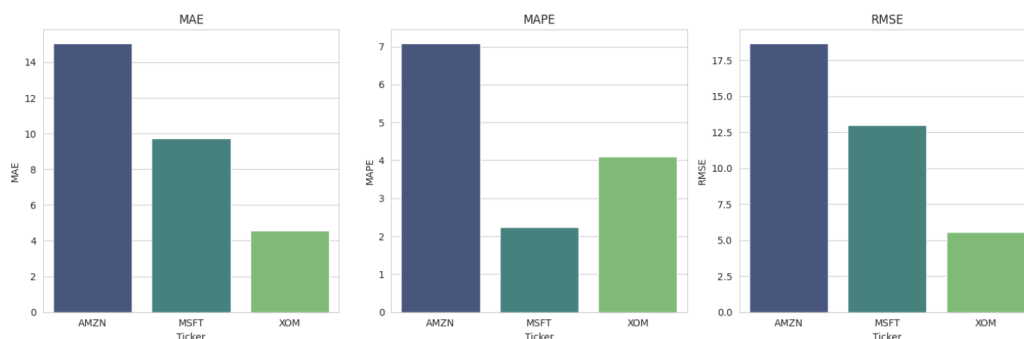


Figure 4.9: Comparison of LSTM 3-day forecast performance for each ticker: (left) MAE, (center) MAPE, (right) RMSE.

4.5.4 Discussion

Both 1-day and 3-day ahead LSTM forecasts had much higher errors than ARIMA or Prophet for all stocks and metrics. The difference was especially clear for more volatile stocks like AMZN and MSFT, and LSTM models did not improve with longer blocks. This shows that, in this walk-forward setup, moderate-sized LSTM models trained on short rolling windows with limited

data may be unstable or may not fit well, and do not make full use of their nonlinear modeling strengths in this context. These results show that model choice and experimental design, such as window size, lookback, and data scaling, are very important for deep learning forecasting of financial time series.

4.6 Transformer Model Results

4.6.1 Model Fitting Procedure

A transformer-based deep learning architecture was applied to benchmark forecasting performance against ARIMA, Prophet, and LSTM. For each ticker, the closing price series was split into training (80%), validation (10%), and test (10%) intervals. At each prediction step, the model was retrained on the most recent 200 days of data using a walk-forward rolling window approach, which mirrors the approach adopted for other models.

Input sequences consisted of 60 consecutive daily closing prices, forming a lookback window for each prediction. For each prediction interval, a new transformer model was instantiated and trained. The architecture comprised two transformer blocks, four-head self-attention, a key dimension of 32, a feed-forward dimension of 64, and a dropout rate of 0.1. The AdamW optimizer was employed with a learning rate of 0.0005 and weight decay of 0.01. Huber loss and early stopping with a patience of three epochs were implemented to mitigate overfitting. All input data underwent MinMax normalization, with the scaler fitted on the combined training and validation sets. Final predictions were inverse-transformed to the original price scale for evaluation.

4.6.2 1-Day Ahead Forecasting

Table 4.6 summarizes the evaluation metrics for 1-day ahead forecasting, and Figure 4.10 presents the actual and predicted trajectories for each ticker.

Ticker	MAE	MAPE (%)	RMSE
AMZN	10.10	4.84	12.61
MSFT	10.03	2.34	12.48
XOM	3.49	3.13	4.53

Table 4.6: 1-day ahead test results for the Transformer

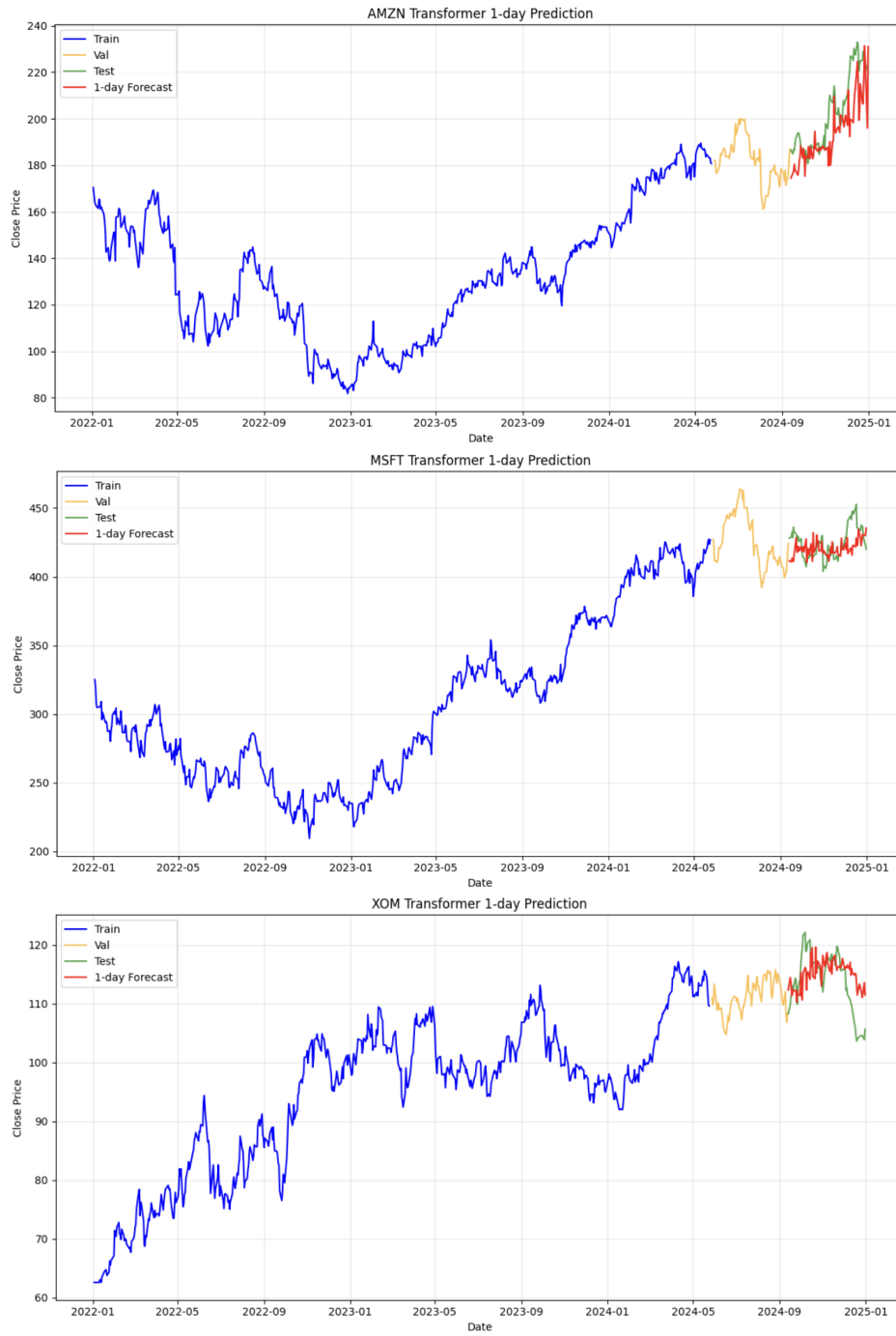


Figure 4.10: 1-day ahead forecasts from the Transformer (Train, Validation, Test, Forecast) for AMZN, MSFT, XOM (top to bottom).

Table 4.6 and Figure 4.10 indicate that the Transformer model outperformed the Long Short-Term Memory (LSTM) network for the highly volatile Amazon (AMZN) ticker in certain instances. However, the Transformer consistently underperformed compared to the Autoregressive Integrated Moving Average (ARIMA) and Prophet models. For Microsoft (MSFT) and ExxonMobil (XOM), the Transformer achieved performance comparable to or marginally better than LSTM, but absolute errors remained higher than those observed for the classical models.

- **AMZN:** MAE 10.10, MAPE 4.84%, RMSE 12.61. Forecasting errors increased significantly during periods characterized by elevated market volatility.
- **MSFT:** MAE 10.03, MAPE 2.34%, RMSE 12.48. While proportional errors remained low, the magnitude of absolute errors was considerable.
- **XOM:** MAE 3.49, MAPE 3.13%, RMSE 4.53. Model performance improved for this relatively stable time series but remained inferior to the results achieved by the ARIMA and Prophet models.

4.6.3 3-Day Ahead Forecasting

The 3-day ahead block forecasting task employed the same architecture and walk-forward protocol. For each forecast block, the model generated three consecutive predictions. The final prediction served as the representative forecast for comparison.

Ticker	MAE	MAPE (%)	RMSE
AMZN	11.61	5.52	14.69
MSFT	11.82	2.76	13.47
XOM	4.42	3.96	5.43

Table 4.7: 3-day ahead test results for the Transformer

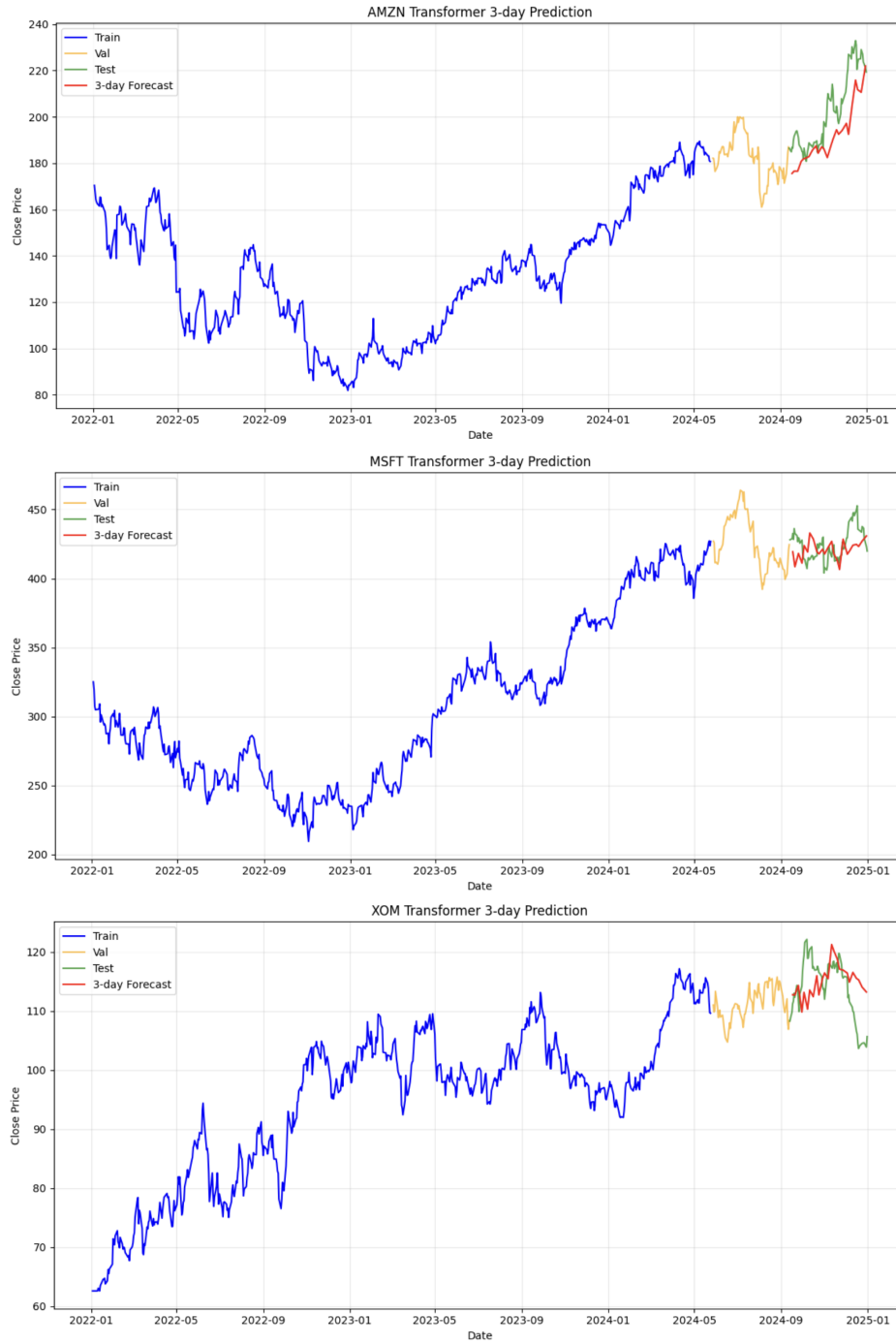


Figure 4.11: 3-day ahead block forecasts from the Transformer (Train, Validation, Test, Forecast) for AMZN, MSFT, XOM (top to bottom).

Error metrics increased slightly for the longer horizon:

- **AMZN:** MAE 11.61, MAPE 5.52%, RMSE 14.69. The model exhibited performance similar to the 1-day forecasting scenario and consistently struggled to track substantial price fluctuations.
- **MSFT:** MAE 11.82, MAPE 2.76%, RMSE 13.47. Error metrics increased slightly compared to the 1-day forecasting results. However, the overall performance pattern was consistent.
- **XOM:** MAE 4.42, MAPE 3.96%, RMSE 5.43. The model demonstrated greater robustness than the LSTM approach, although it remained less effective than the statistical method.

4.6.4 Discussion

Across both 1-day and 3-day forecasting horizons, the Transformer model outperformed LSTM in certain cases but did not match the predictive accuracy of the ARIMA or Prophet models within this experimental framework. The Transformer exhibited the greatest performance deficits when applied to highly volatile financial assets and during multi-step forecasting, where prediction errors accumulated. These results indicate that although attention mechanisms offer some benefits, the Transformer’s performance remains constrained without extended training periods, enhanced exogenous variable integration, or access to larger datasets. Overall, these findings underscore the persistent difficulties associated with applying deep learning methods to financial time series forecasting in realistic walk-forward experimental contexts.

4.7 Prophet Model Results

4.7.1 Model Fitting Procedure

The Prophet model was applied to the log-transformed closing prices of each ticker to address non-stationarity and maintain positive predictions. The dataset was divided into training (80%), validation (10%), and test (10 %) subsets. Walk-forward forecasting was implemented using a rolling window of 200 observations. The model employed linear growth and was optimized with the following hyperparameters: changepoint prior scale of 0.5, seasonality prior scale of 10.0, and holidays prior scale of 5.0. Custom seasonalities were specified as monthly (period 30.5, Fourier order 5) and quarterly (period 91.25, Fourier order 3). Weekly seasonality was included, while daily and yearly seasonalities were excluded. United States major holidays for 2022 to 2024 were incorporated. Model predictions were generated on the log scale and then exponentiated to the original scale for evaluation.

4.7.2 1-Day Ahead Forecasting

The 1-day ahead forecasting performance of the Prophet model was evaluated for Amazon (AMZN), Microsoft (MSFT), and ExxonMobil (XOM) using a rolling-window approach with a length of 200 observations. Figure 4.12 presents the forecast plots.

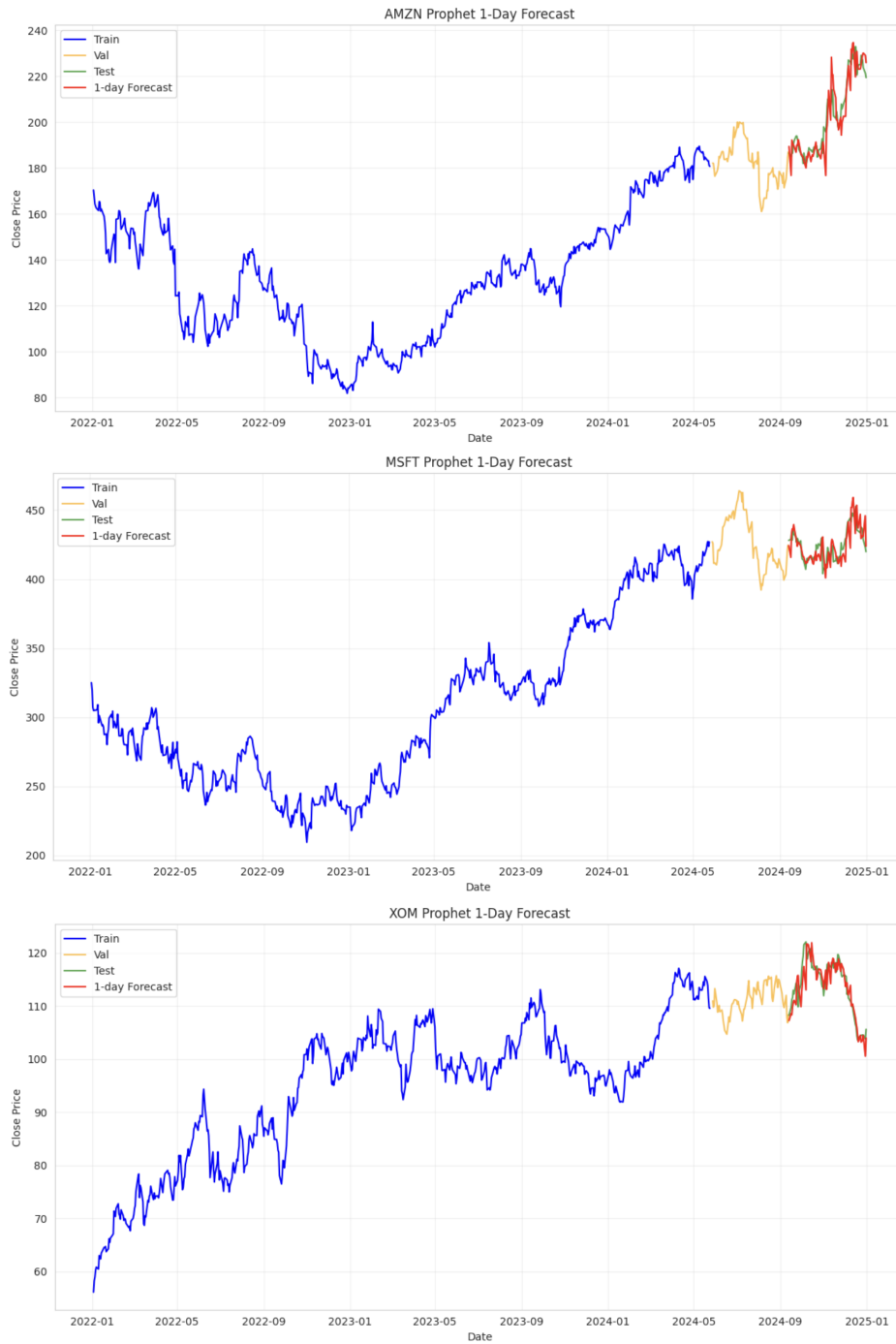


Figure 4.12: Prophet 1-day ahead forecasts (Train, Validation, Test, Forecast) for (top) AMZN, (middle) MSFT, and (bottom) XOM.

The forecast curves (red) closely tracked the short-term trends of the actual stock prices (green). The model configuration enabled effective capture of seasonal patterns and holidays. The forecasts exhibited smoothing, especially during periods of high volatility.

Ticker	MAE	MAPE (%)	RMSE
AMZN	4.49	2.19	6.06
MSFT	6.04	1.42	8.34
XOM	1.84	1.60	2.38

Table 4.8: Test set performance of Prophet 1-day forecasts

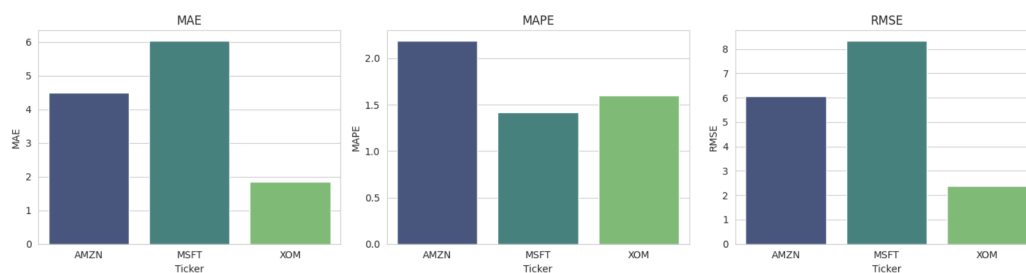


Figure 4.13: Comparison of Prophet 1-day forecast performance for each ticker: (left) MAE, (center) MAPE, (right) RMSE.

As summarized in Table 4.8 and Figure 4.13, key findings are:

- **AMZN (Amazon):** MAE 4.49, MAPE 2.19%, RMSE 6.06. The model demonstrated moderate robustness to volatility. However, error rates increased in response to rapid price fluctuations.
- **MSFT (Microsoft):** MAE 6.04, MAPE 1.42%, RMSE 8.34. The model maintained high proportional accuracy, although absolute error values were influenced by fluctuations characteristic of the technology sector.

- **XOM (ExxonMobil):** MAE 1.84, MAPE 1.60%, RMSE 2.38, benefiting from the energy sector’s relative stability.

4.7.3 3-Day Ahead Forecasting

The 3-day ahead block forecasts were generated using the Prophet model for Amazon (AMZN), Microsoft (MSFT), and ExxonMobil (XOM). The configuration matched the 1-day forecasting approach, including log transformation and parameter tuning. The walk-forward mechanism produced forecasts for 3-day blocks, using the final day’s value as the prediction and updating the historical data with actual values. The resulting forecast plots are presented in Figure 4.14.



Figure 4.14: Prophet 3-day ahead block forecasts (Train, Validation, Test, Forecast) for (top) AMZN, (middle) MSFT, and (bottom) XOM.

The forecasted data points (red) followed the overall trend of the actual stock prices (green). However, the divergence between forecasted and actual values increased during periods of high market volatility relative to the 1-day forecast results.

Ticker	MAE	MAPE (%)	RMSE
AMZN	6.95	3.32	9.05
MSFT	8.60	2.02	11.28
XOM	2.59	2.26	3.27

Table 4.9: Test set performance of Prophet 3-day forecasts

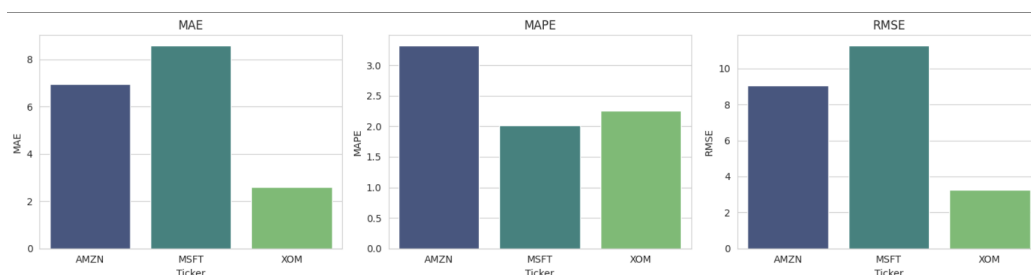


Figure 4.15: Comparison of Prophet 3-day forecast performance for each ticker: (left) MAE, (center) MAPE, (right) RMSE.

As summarized in Table 4.9 and Figure 4.15, key findings are:

- **AMZN (Amazon):** MAE 6.95, MAPE 3.32%, RMSE 9.05. The increase in errors is attributable to the accumulation of predictive uncertainty across the multi-day forecasting horizon.
- **MSFT (Microsoft):** MAE 8.60, MAPE 2.02%, RMSE 11.28. The forecasting model effectively identified general trends but exhibited reduced accuracy during periods of sustained market volatility.
- **XOM (ExxonMobil):** MAE 2.59, MAPE 2.26%, RMSE 3.27. Model

performance degraded less severely in the comparatively stable energy sector.

Comparing the 1-day and 3-day results, the 3-day forecasts exhibited higher errors across all metrics, with average increases of approximately 48% in MAE, 49% in MAPE, and 41% in RMSE. This highlights Prophet’s strength in capturing seasonality for short horizons but limitations in longer-term predictions for volatile stocks like AMZN and MSFT, while performing relatively better for stable series like XOM.

4.8 Comparative Results

This section provides a comprehensive comparison of the forecasting performance across all models (ARIMA, Prophet, LSTM, and Transformer) for both 1-day and 3-day ahead horizons. Results are analyzed by ticker (AMZN, MSFT, XOM) using the evaluation metrics MAE, MAPE, and RMSE. Tables summarize the errors for each model and horizon, while bar charts visualize the comparisons to highlight relative strengths and weaknesses.

4.8.1 1-Day Ahead Comparison

Table 4.10 presents the 1-day ahead forecasting errors for all models across the three tickers.

Ticker	MAE				MAPE (%)				RMSE			
	ARIMA	Prophet	LSTM	Transformer	ARIMA	Prophet	LSTM	Transformer	ARIMA	Prophet	LSTM	Transformer
AMZN	2.87	4.49	14.10	10.10	1.41	2.19	6.67	4.84	3.66	6.06	17.38	12.61
MSFT	3.75	6.04	9.71	10.03	0.88	1.42	2.25	2.34	5.46	8.34	12.40	12.48
XOM	1.03	1.84	4.10	3.49	0.90	1.60	3.68	3.13	1.37	2.38	5.22	4.53

Table 4.10: Comparative 1-day ahead forecasting errors by model and ticker

ARIMA consistently achieved the lowest errors across all metrics and tickers, followed closely by Prophet. Deep learning models (LSTM and Transformer) exhibited higher errors, particularly for volatile stocks like AMZN.

For instance, ARIMA’s MAE for AMZN was 2.87, compared to 14.10 for LSTM.

4.8.2 3-Day Ahead Comparison

Table 4.11 summarizes the 3-day ahead block forecasting errors.

Ticker	MAE				MAPE (%)				RMSE			
	ARIMA	Prophet	LSTM	Transformer	ARIMA	Prophet	LSTM	Transformer	ARIMA	Prophet	LSTM	Transformer
AMZN	4.88	6.95	15.07	11.61	2.37	3.32	7.09	5.52	5.79	9.05	18.70	14.69
MSFT	6.39	8.60	9.73	11.82	1.51	2.02	2.25	2.76	8.37	11.28	12.99	13.47
XOM	2.16	2.59	4.56	4.42	1.90	2.26	4.09	3.96	2.67	3.27	5.59	5.43

Table 4.11: Comparative 3-day ahead forecasting errors by model and ticker

Prediction errors increased for all models over the 3-day forecasting horizon. Both ARIMA and Prophet continued to outperform the neural network models. The LSTM network exhibited the greatest performance decline, particularly for Amazon (AMZN), where MAE reached 15.07 compared to ARIMA’s 4.88.

Table 4.12 shows the percentage increase in metrics from 1-day to 3-day predictions for each model and ticker.

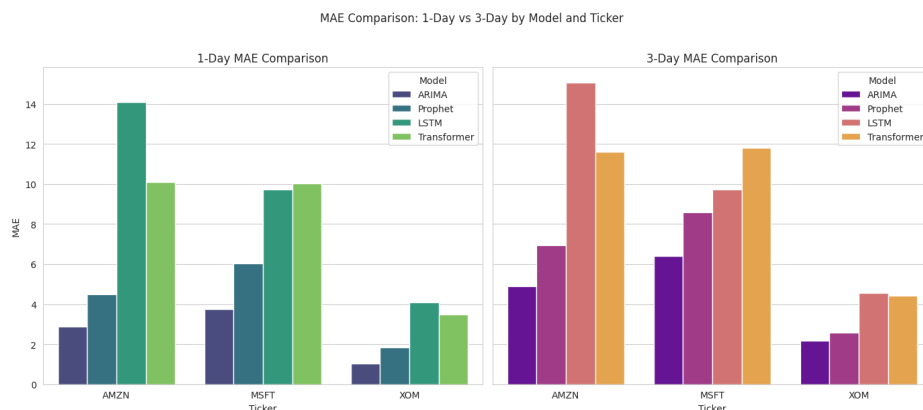


Figure 4.16: Comparative MAE by ticker, model, and forecast horizon (1-day and 3-day).

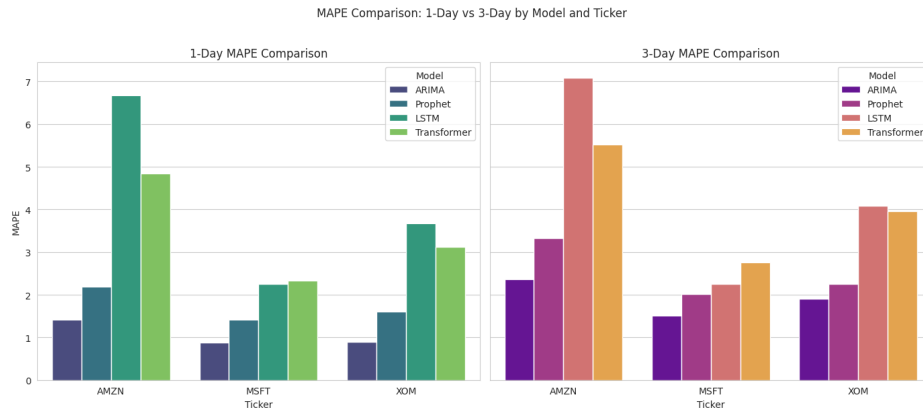


Figure 4.17: Comparative MAPE by ticker, model, and forecast horizon (1-day and 3-day).

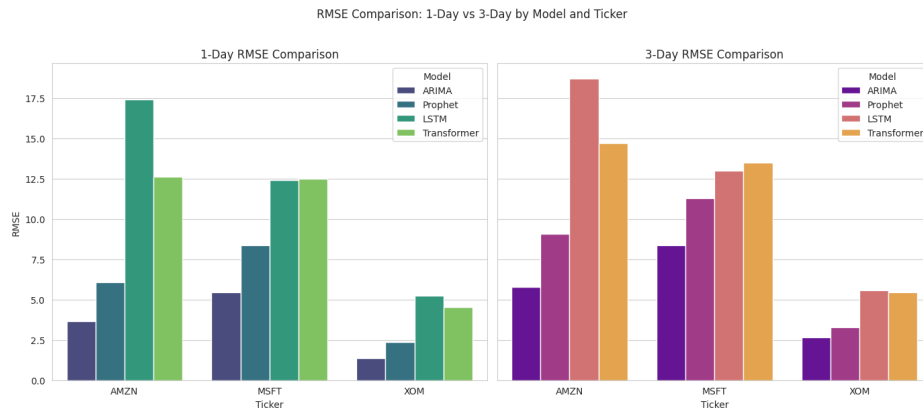


Figure 4.18: Comparative RMSE by ticker, model, and forecast horizon (1-day and 3-day).

Ticker	MAE Increase (%)				MAPE Increase (%)				RMSE Increase (%)			
	ARIMA	Prophet	LSTM	Transformer	ARIMA	Prophet	LSTM	Transformer	ARIMA	Prophet	LSTM	Transformer
AMZN	70.03	54.79	6.88	14.95	68.09	51.60	6.30	14.05	58.20	49.34	7.59	16.50
MSFT	70.40	42.38	0.21	17.85	71.59	42.25	0.00	17.95	53.30	35.25	4.76	7.93
XOM	109.71	40.76	11.22	26.65	111.11	41.25	11.14	26.52	94.89	37.39	7.09	19.87

Table 4.12: Percentage increase in metrics from 1-day to 3-day predictions by model and ticker

4.8.3 Discussion

Statistical models such as ARIMA and Prophet generally outperformed deep learning approaches, emphasizing their robustness for financial forecasting with limited data. Sector volatility played a key role. XOM had lower errors across models, while AMZN highlighted neural networks' limitations.

Analysis of the percentage increases in error metrics from 1-day to 3-day horizons (Table 4.12) demonstrates varying sensitivity to forecast extension. Traditional models such as ARIMA exhibited the highest relative degradations. For example, MAE increased by more than 70% for AMZN and MSFT, and over 100% for XOM. This indicates that these models lose precision quickly with multi-step forecasts, likely because their assumptions accumulate over time, regardless of whether the time series is stable or volatile. Prophet had moderate MAE increases, between 40% and 55%, which means it handles longer forecasts better but is still affected by the forecast length. Deep learning models had smaller increases. LSTM's MAE rose by less than 12% for all tickers, and Transformer models ranged from 15% to 27%. This is partly because their 1-day forecast errors were already high, so the error did not grow as much in percentage terms.

In summary, neural networks have a higher overall error rate but perform more consistently than traditional models in long-term predictions. This is especially clear for volatile stocks like AMZN, where LSTM's MAE only increased by 6.88%. These results demonstrate the strengths and weaknesses of statistical models and deep learning models. In conclusion, statistical models are more accurate in short-term predictions but their accuracy declines rapidly in long-term predictions, while deep learning models show no significant difference in accuracy between short-term and long-term predictions.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This thesis conducted a comparative analysis of ARIMA, LSTM, Transformer, and Prophet models to predict daily closing prices of AMZN, MSFT, and XOM stocks, using data from January 1, 2022, to January 1, 2025. The study evaluated 1-day and 3-day ahead performance using a walk-forward validation approach and performance metrics such as MAE, MAPE, and RMSE.

Traditional statistical models, especially ARIMA, recorded the lowest forecasting errors for all stocks and prediction periods. These results are consistent with previous studies such as Spulbar et al. [6] and Ahammad et al. [7], which demonstrated ARIMA’s reliability under stable market conditions. The model was particularly effective on the XOM dataset, which showed low volatility and a more linear trend. Prophet also performed well, especially with shorter time ranges and stocks with low volatility. These results align with studies by Kolari et al. [5] and Kakinuma [15], which emphasized Prophet’s strength in capturing trends and seasonality, particularly in small datasets or when outliers are present.

In contrast, deep learning models like LSTM and Transformer, which are

theoretically powerful and supported by studies by Mroua et al. [9] and Wang et al. [13], showed relatively lower performance in this study. The difference may be due to data scarcity, as both models generally require large volumes of data and careful hyperparameter tuning. However, when analyzing the increase in error rates from 1-day to 3-day forecasts, the neural network model showed higher consistency than traditional models. For instance, the MAE of LSTM increased by only 6.88% for AMZN, while the MAE of ARIMA increased by over 70%. This indicates that statistical models are more accurate for short-term predictions but their performance deteriorates sharply as the prediction period lengthens.

Overall, these results emphasize the balance between statistical models and neural network models. Traditional approaches such as ARIMA and Prophet remain powerful for short-term predictions, stable predictions, and small datasets. On the other hand, LSTM and Transformer models demonstrate stable performance in long-term predictions but have higher baseline errors. Therefore, when selecting a suitable model for financial time series prediction, it is essential to carefully consider factors such as the scope of the data, volatility, and prediction period.

5.2 Future Work

Future research can expand on this study by incorporating external factors, such as macroeconomic indicators, interest rates, GDP, and sentiment data from news and social media. These additions may make models more robust, particularly for deep learning applications. Combining statistical methods, such as ARIMA residuals, with neural networks could also help improve accuracy in volatile markets by leveraging the strengths of both approaches.

Exploring longer forecast periods, like 7-day or 30-day intervals, or using real-time streaming data could help show how well the models scale. Including more tickers from different sectors or global indices and trying advanced

models such as Temporal Fusion Transformers or ensemble methods may help confirm and improve these findings. Using uncertainty quantification tools, such as Bayesian Prophet or probabilistic LSTMs, could also make risk assessments more reliable in real-world finance.

Bibliography

- [1] Shi Z. MambaStock: Selective state space model for stock prediction; 2024. Available from: <https://arxiv.org/abs/2402.18959>.
- [2] Xiao J, Deng T, Bi S. Comparative Analysis of LSTM, GRU, and Transformer Models for Stock Price Prediction; 2024. Available from: <https://arxiv.org/abs/2411.05790>.
- [3] Gupta H, Jaiswal A. A Study on Stock Forecasting Using Deep Learning and Statistical Models; 2024. Available from: <https://arxiv.org/abs/2402.06689>.
- [4] Li T, Liu Z, Shen Y, Wang X, Chen H, Huang S. MASTER: Market-Guided Stock Transformer for Stock Price Forecasting; 2023. Available from: <https://arxiv.org/abs/2312.15235>.
- [5] Kolari JW, Sanz I. Forecasting Bank Capital Ratios Using the Prophet Model by Facebook. SSRN Electronic Journal. 2022.
- [6] Spulbar C, Ene CC. Predictive Analytics in Finance Using the Arima Model. Application for Bucharest Stock Exchange Financial Companies Closing Prices. Studies in Business and Economics. 2024 12;19:30-49.
- [7] Ahammad I, Sarkar WA, Meem FA, Ferdus J, Ahmed MK, Rahman MR, et al. Advancing Stock Market Predictions with Time Series Analysis including LSTM and ARIMA. Cloud Computing and Data Science. 2024 05:226-41.

- [8] Fischer T, Krauss C. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*. 2018 10;270:654-69.
- [9] Mroua M, Lamine A. Financial Time Series Prediction Under COVID-19 Pandemic Crisis with Long Short-Term Memory (LSTM) Network. *SSRN Electronic Journal*. 2022.
- [10] Nalabola P. Stock Price Prediction Using Long Short-Term Memory Networks. 2025 01. Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5332359.
- [11] osman. Integrating Deep Learning and Econometrics for Stock Price Prediction: An Empirical Study of Lstm and Traditional Time Series Models. *Ssrncom*. 2025. Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5340299.
- [12] Nguyen A, Ha S. A Lightweight Multi-Head Attention Transformer for Stock Price Forecasting. *SSRN Electronic Journal*. 2024.
- [13] Wang C, Chen Y, Zhang S, Zhang Q. Stock market index prediction using deep Transformer model. *Expert Systems with Applications*. 2022 12;208:118128.
- [14] Zhang H. Stock price forecast based on improved Transformer. 2024 01:170-82. Available from: <https://www.atlantis-press.com/proceedings/icaid-24/126002614>.
- [15] Kakinuma Y. Forecasting Financial Innovation Fund Prices: Machine Learning Approach. *Social Science Research Network*. 2023 01.
- [16] Hakkar F. Intelligent Forecasting of GBP/USD Exchange Rates: A Comparative Analysis of ARIMA, Prophet, and LSTM Models. 2025. Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5310370.

- [17] Bryzgalova S, Lerner S, Lettau M, Pelger M. Missing Financial Data. SSRN Electronic Journal. 2022.
- [18] Htun HH, Biehl M, Petkov N. Survey of feature selection and extraction techniques for stock market prediction. Financial Innovation. 2023 01;9.
- [19] Adams JC, Hayunga DK, Mansi S, Reeb DM. Identifying Outliers in Finance. SSRN Electronic Journal. 2017.
- [20] Box GEP, Jenkins GM, Reinsel GC, Ljung GM. Time series analysis : forecasting and control. John Wiley Sons; 2015.
- [21] Dickey DA, Fuller WA. Distribution of the Estimators for Autoregressive Time Series with a Unit Root. Journal of the American Statistical Association. 1979 06;74:427-31. Available from: <https://www.tandfonline.com/doi/abs/10.1080/01621459.1979.10482531>.
- [22] Hochreiter S, Schmidhuber J. Long Short-Term Memory. Neural Computation. 1997 11;9:1735-80. Available from: <https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory?redirectedFrom=fulltext>.
- [23] Yan S. Stock Price Prediction and Investment Simulation - Based on SSA-CNN-LSTM Method; 2024. Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5120133.
- [24] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al.. Attention Is All You Need; 2017. Available from: <https://arxiv.org/abs/1706.03762>.
- [25] Lim B, Arik SO, Loeff N, Pfister T. Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting. arXiv:191209363 [cs, stat]. 2020 09. Available from: <https://arxiv.org/abs/1912.09363>.

- [26] Wu N, Green B, Ben X, O'Banion S. Deep Transformer Models for Time Series Forecasting: The Influenza Prevalence Case. arXiv:200108317 [cs, stat]. 2020 01. Available from: <https://arxiv.org/abs/2001.08317>.
- [27] Taylor SJ, Letham B. Forecasting at scale. Forecasting at scale. 2017 09. Available from: <https://peerj.com/preprints/3190/>.
- [28] anon J, Sakowski P, Ślepaczuk R. Mean Absolute Directional Loss as a New Loss Function for Machine Learning Problems in Algorithmic Investment Strategies. SSRN Electronic Journal. 2023.
- [29] Rao J, Wang Z. Optimizing Stock Market Return Forecasts with Uncertainty Sentiment: Leveraging LLM-based Insights. 2025 01. Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5117562.
- [30] Kim S, Kim H. A new metric of absolute percentage error for intermittent demand forecasts. International Journal of Forecasting. 2016 07;32:669-79.